Llewy Smith
a1706836

Assessment 1C: Big Data Analysis

Table of Contents

Part 1: Problem Description (3 marks)

The code and outputs referred to in this report are available via my public GitHub repository. My complete set of research questions are available in Appendix (1) of this report.

Project Summary

My project investigates whether we can identify students at risk of failing a course prior to their Census Date (approximated as 25% of the way through the course). I test this using the set of publicly available Open University Learning Analytics Datasets, known as OULAD. The input features I gather from OULAD span four broad categories:

(1) learning analytics (Virtual Learning Environment and assessments)
(2) demographic
(3) administrative
(4) course metadata

The class distribution is moderately imbalanced with 41% of courses resulting in fail or withdraw grades (which I group as failed), motivating my use of evaluation metrics (e.g. F1 score) and sampling strategies (e.g. SMOTE) that better manage this imbalance. To mimic real-world practicality, I truncate all temporal features at 25% of each course's teaching term. This ensures my model only sees what advisors would realistically know at this stage of the course.

Data tables in OULAD

Table 1: Data Tables in OULAD (OULAD, 2017)

Table Name	Records	Description
courses	22	Metadata including code, schedule, and length
assessments	2,913	Details such as type, weighting, and due date
vle	6,389	Virtual learning environment (VLE) activities
studentInfo	32,504	Demographic and outcome data for each student-course enrolment
studentRegistration	32,504	Dates of enrolment and unenrolment
studentAssessment	26,699	Submissions, scores, and dates
studentVLE	10.6 million	VLE interactions by student, activity, and date

Key features:

A full data schema (including written descriptions and a diagram) is available publicly on the OULAD (2017) webpage, accessible here.

Learning analytics (VLE engagement and assessment)

This data is sourced from studentVLE.csv, studentAssessment.csv, vle.csv and assessments.csv. Examples are:

- **sum_click** - the aggregate of interactions each student has with material each day
- **date_submitted** – the date the assessment is submitted, measured as the number of days since the start of the course

I also construct aggregates and threshold based dummy variables using these so that my CNN part can learn from these. Examples are sum_click (clicks over the first 25% of the course), early_weighted_avg_score (of assessments over 25% of the course), and an early_failed_assessment (flag for 1 or more failed assessments).

Demographic features

This data is sourced from studentInfo.csv. Examples are:

- **highest_education** – student's highest education level before taking this course
- **imd_band** - calculated Index of Multiple Depravation band using student's address

These features are static. They are Boolean (e.g. disability), ordinal categorical (e.g. imd_band) or nominal categorical (e.g. region). I encode these using Skikit-learn's OneHotEncoder in my pre-processing pipeline.

Administrative features

This data is sourced from studentInfo.csv and studentRegistration.csv. Examples include:

- **num_of_prev_attempts** - aggregate of student's previous attempts at the course
- **date_registration** - date of student's course enrolment relative to start date
- when they enrolled, relative to the module start

These features help identify academic overload (e.g., high credit load), prior struggle (e.g., repeat enrolments), and possible delayed start (e.g., late enrolment).

Course metadata

This data is sourced from courses.csv and assessments.csv. It gives fixed characteristics about each course. Examples include:

- **code_module** - course name (e.g., GGG, DDD)
- **code_presentation** - semester of delivery (e.g., 2013B or 2014J)
- **module_presentation_length** - total length of the course in days

These features are used to calculate the 25% cutoff point of each course. They also unlock nuance at the course level - some courses may set more assessment early on, while others may rely more on VLE clicks.

Input/Output Summary:

Inputs: 34 engineered features remained after preprocessing. Recursive Feature Elimination (RFE) was tested in CNNs, and indeed they performed best with RFE using k=10.

Outputs: Binary course outcome (failed_course, 1 = fail and 0 = pass). All models use the same stratified train/test split (80/20, random_state=42).

IDs retained: enrolment_instance_id was preserved outside the model matrix to allow student-level explanations and reporting (e.g., in the LIME interface). This allowed me to run a program which successfully delivers predictions and explainability at an individual level.

Part 2: Data Pre-processing (2 marks)

After loading the 7 OULAD tables, I first cut every temporal signal at 25% of each course's length (my proxy for Census Date). This allows me to then build aggregate features that both advisors and predictive models can derive rich information from. The key steps I followed are:

1. **add_early_cutoff_to_courses** tags the 25% cutoff day for each course presentation.
2. **build_aggregated_vle_features** and **build_aggregated_assessment_features** each collapse millions of log rows into early-period indicators, such as **sum_click**, **days_clicked**, and **early_weighted_avg_score**.
3. Create a binary **early_failed_assessment** flag to identify students who failed any early assessments.
4. Merge engineered and other features using the **assemble_aggregated_master_table** function.
5. Encode and clean data using **encode_and_clean** and **tidy_columns** functions.
6. Split the data using **train_test_split(..., stratify=y, random_state=42),** while keeping **enrolment_instance_id** out of the model matrix. ID is preserved only for my application interpretation using my Local Interpretable Model-agnostic Explanation (LIME) interface.

I apply preprocessing through a single **ColumnTransformer**: numeric columns are standardised using **StandardScaler()**, while categorical variables (e.g., region, module code, presentation) are one-hot encoded with **OneHotEncoder(drop='first', handle_unknown='ignore')**. This produces a dense, scaled matrix suitable for Logistic Regression, the MLP and the CNN. Missing values are minimal after aggregation; when an event is absent (e.g., no early assessment or no clicks), a zero is valid and so I manually imputed these into my aggregates after testing. No other static features had missing values, so an explicit imputer (e.g. Skikit-learn's SimpleImputer or KNNImputer) was not required.

Because the outcomes are moderately imbalanced (approximately 41% of enrolments result in failure or withdrawal), I apply Synthetic Minority Over-sampling Technique (SMOTE) on the training fold (via my **oversample_smote** function) to boost the number of minority (fail class) examples. I do this after scaling and encoding, and never apply it to the test set, ensuring evaluation remains fair. I also measure my models' performance with and without SMOTE for comparison, and indeed the SMOTE models performed considerably better (discussed later).

To reduce feature noise and potentially enable simpler LIME explanations, I apply Recursive Feature Elimination (RFE) using a Logistic Regression estimator (**apply_rfe** function). I loop over values of k in the set {3, 5, 10, 15, 20, 25, 30, 34} to identify how many features to retain and to test which dimensionality works best for the CNN in a similar fashion to a GridSearch.

Part 3: Model Selection (3 marks)

I compare four predictive models in Part C:
1. Logistic Regression (LR),
2. Random Forest (RF),
3. Multilayer Perceptron (MLP), and
4. 1D Convolutional Neural Network (CNN).

Each model is trained on the same stratified 80/20 train-test split, using identical preprocessing pipelines and input features to ensure comparisons reflect differences in model capability, not data treatment. However, given the goals of my project, I invested significantly more time developing the CNN model. This is because my project was intended as an extension of Adefemi & Mutanga's (2025) research paper rather than a fresh start. Over the course of the project, I found inconsistencies in their work and chose to pivot - hence, I do not present a hybrid CNN-LSTM or GRU with SHAP explainability as I originally planned. These are general reasons for using the models, however realistically I chose them also because of their success in previous research papers such as Adefemi & Mutanga (2025),

Logistic Regression
- A baseline linear classifier well-suited to high-dimensional, one-hot encoded data.
- Transparent and easy to interpret, with coefficients directly corresponding to features.
- Serves as a benchmark for evaluating more complex models.

Random Forest
- A robust ensemble of decision trees that handles both categorical and numerical data.
- Captures non-linear feature interactions and provides internal estimates of feature importance.
- Less transparent than LR but generally strong in tabular data scenarios.

Multilayer Perceptron (MLP)
- A shallow neural network with non-linear activation that models complex feature relationships.
- Rapid training and capable of generalising well with dropout and L2 regularisation.
- Slightly less interpretable than tree-based models without more complex XAI methods.

1-D Convolutional Neural Network (CNN)
- Exploits local feature patterns as ordered by Recursive Feature Elimination (RFE).
- Contains fewer parameters than recurrent models, offering a trade-off between complexity and generalisation.
- Compatible with LIME for explainability.

With it as my intended main model - I developed the CNN architecture iteratively through testing, with early stopping implemented to avoid overfitting. The architecture includes convolutional and dense layers with ReLU activations, followed by dropout regularisation and a final sigmoid layer for binary classification. This structure balances capacity with generalisability - a useful trait given my relatively small input dimensionality (RFE: k = 10-15).

My CNN was never intended to be state-of-the-art in predictive power; more of my focus was on ensuring the model could be made explainable, which it successfully was.

Part 4: Model Refinement (4 marks)

Most of my model tuning and my explainability tuning centred on the CNN, as that was my focus model. I tested 8 different values of RFE feature counts (k in the set {3, 5, 10, 15, 20, 25, 30, 34}) and compared results both with and without SMOTE. This sweep was done using the same model architecture and input pipeline, with test set performance logged after each run. You can see the results in the RFE/SMOTE comparison chart (Figure 2), where both F1 and PR AUC peak around 10 features when using SMOTE. Without SMOTE, performance is less and marginally less stable - especially in F1.

Figure 2: CNN Performance: RFE Feature Counts and SMOTE Impact

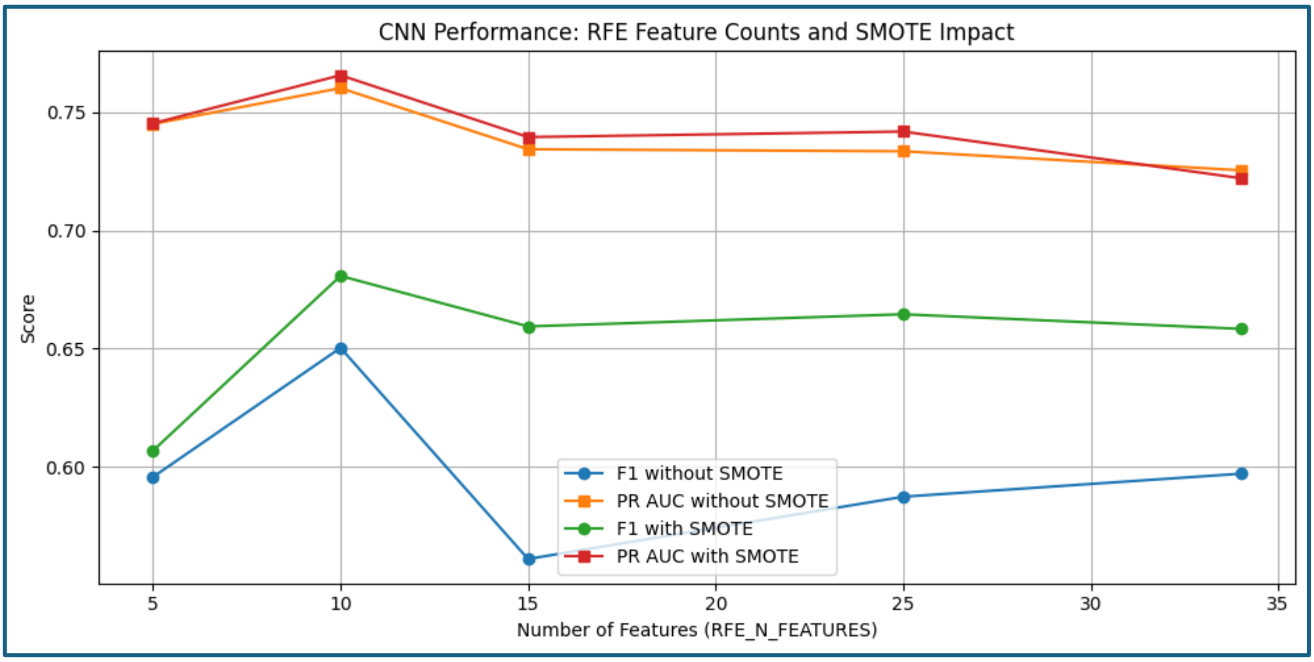


Figure 2 (and the associated code outputs) gave me three clear takeaways.

1. SMOTE consistently helped the CNN improve F1 score as intended. A corresponding confusion matrix showed that this was due to SMOTE models having higher recall on the failure class. This is expected as SMOTE counters class imbalance.
2. Trimming input noise with RFE made a big difference. Around 10 features gave the best generalisation and cleanest LIME explanations later. I selected 10 as the final k.
3. Although not shown in Figure 2 - training time differences matter:
    a. When capped at 2 epochs, both RFE and SMOTE are hugely important and are the difference between a model performing poorly or well
    b. When capped at 20 epochs (as in Figure 2), SMOTE is still important but there is not as huge a difference in performance between an RFE (k=10) and (k=max=34)

This table (Figure 2) helped me land on my best model (explained later), which I then used to tune my explainability tools.

Part 5: Performance Description (3 marks)

To reflect the goal of early identification of at-risk students, I prioritised F1 score (Fail class) and Precision Recall (PR) AUC over traditional accuracy. This is because false positives (flagging a student as at-risk when they're not) are less damaging than false negatives (missing a student who needs support).

- **F1 (Fail)** balances precision and recall -ideal for triaging interventions fairly.
- **PR AUC** better reflects performance under class imbalance (~41% fail rate).

Accuracy and ROC AUC were logged but not relied on - both are inflated by the dominant Pass class.

Table 3: model performance on validation set:

Model	Accuracy	Precision	Recall	F1	ROC AUC	PR AUC
Logistic Regression	0.74	0.68	0.69	0.68	0.81	0.78
Random Forest	0.74	0.70	0.66	0.68	0.80	0.77
MLP	0.75	0.75	0.58	0.65	0.81	0.78
CNN	0.71	0.64	0.70	0.68	0.79	0.75

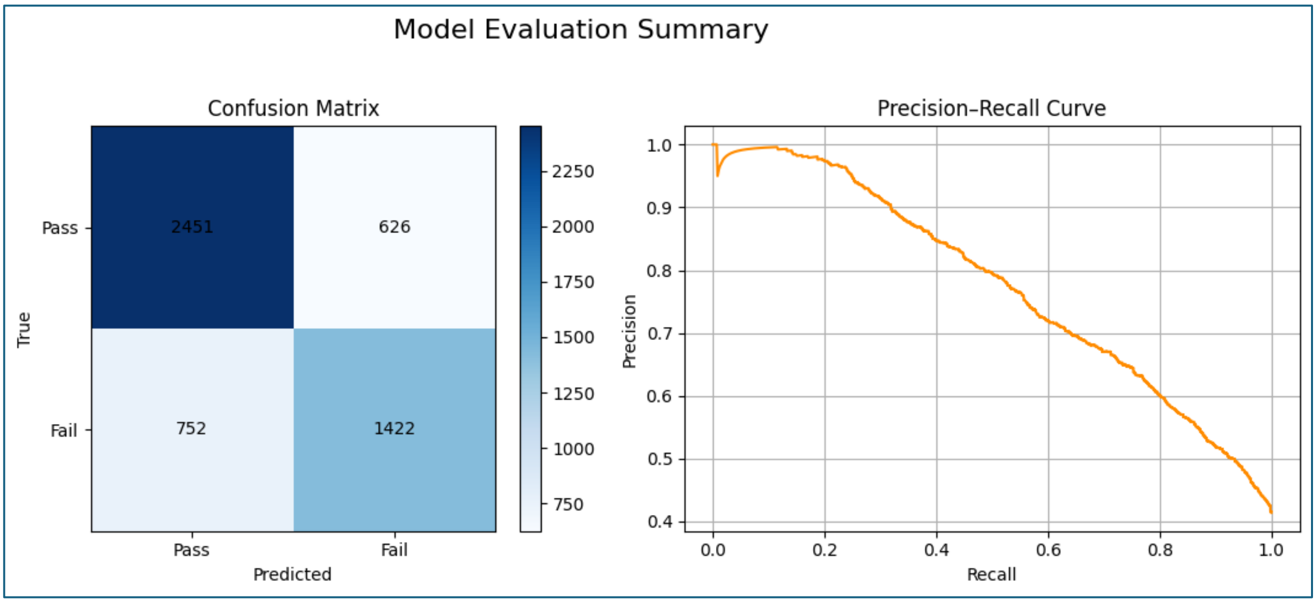
The marginal difference in F1 and PR AUC between LR and RF suggests either could serve as a strong candidate for production use, but other considerations (such as explainability) informed the final recommendation. My CNN model delivered consistent gains from SMOTE, and its F1 peaked when using 10 RFE-selected features. These performance gains are shown clearly in the plots included earlier (Figures 2-3). Despite this, the simple logistic regression model was the highest performer considering both chosen metrics - F1 and PR AUC.

This final CNN had a test F1 of 0.68 (fail class), with PR AUC of 0.75.

<u>Best CNN Model:</u>
---- RFE_N_FEATURES = 10, SMOTE = True ----
CNN → F1: 0.681, PR_AUC: 0.779

Figure 4 shows the confusion matrix and PR curve. False positives are still common, but precision stays above 70% across most recall levels. Subjectively, this makes the model usable as an advisory tool. Refinement here meant choosing settings that made predictions stable and interpretable, not just accurate.

Figure 4: Best CNN Model - Confusion Matrix and PR Curve
RFE: k=10, SMOTE: True



Part 6: Results Interpretation (5 marks)

Best model

Logistic Regression and Random Forest tied on F1 (0.68), but LR had a slight edge in PR AUC and is easier to interpret. While the CNN and MLP offered unique architectural advantages, they did not outperform LR or RF in core metrics.

Logistic Regression is recommended as my production model. Random Forest remains a strong alternative for future testing, and my CNN continues as my preference for future testing.

The final CNN model was not the best overall but was able to be applied in a way that supports direct human intervention in the next section. It is in a viable state to be used for proactive advising aligned with the Support for Students policy as discussed in Part A (University of Adelaide, 2023).

Explainability in action using LIME

Here are a few cases of its application at the student level in my code. I made a program that was able to use LIME explainability (as opposed to the original goal of SHAP - which proved troublesome in Keras 3 using and using CNN or any deep learning models). These were a huge success.

Each LIME explanation shows the exact features that pushed a prediction towards ‘Fail’ or ‘Pass’ for a given student. For example, in Figure 5, on student was flagged at 80% of failing by the model. This student did indeed fail. At the 25% mark of the course, an academic advisor would have been able to use this model to (1) identify this student and their predicted level of risk and (2) see that the biggest contributory factor to this was the student’s early weighted average score of assessments, which was 0.53 standard deviations below the average. Figure 5 is on the next page.

Figure 5: LIME application #1

```
Student Index: 595
Enrolment Instance ID: 535794_DDD_2013J
Predicted: Fail (80% probability of fail)
Actual:    Fail

Top Contributing Features:
- early_weighted_avg_score: -0.53 (↑ Predictor of Fail)
- code_module_GGG: 0.00 (↑ Predictor of Fail)
- code_module_CCC: 0.00 (↓ Predictor of Pass)
- code_presentation_2014J: 0.00 (↑ Predictor of Fail)
- days_clicked: -0.10 (↓ Predictor of Pass)
```

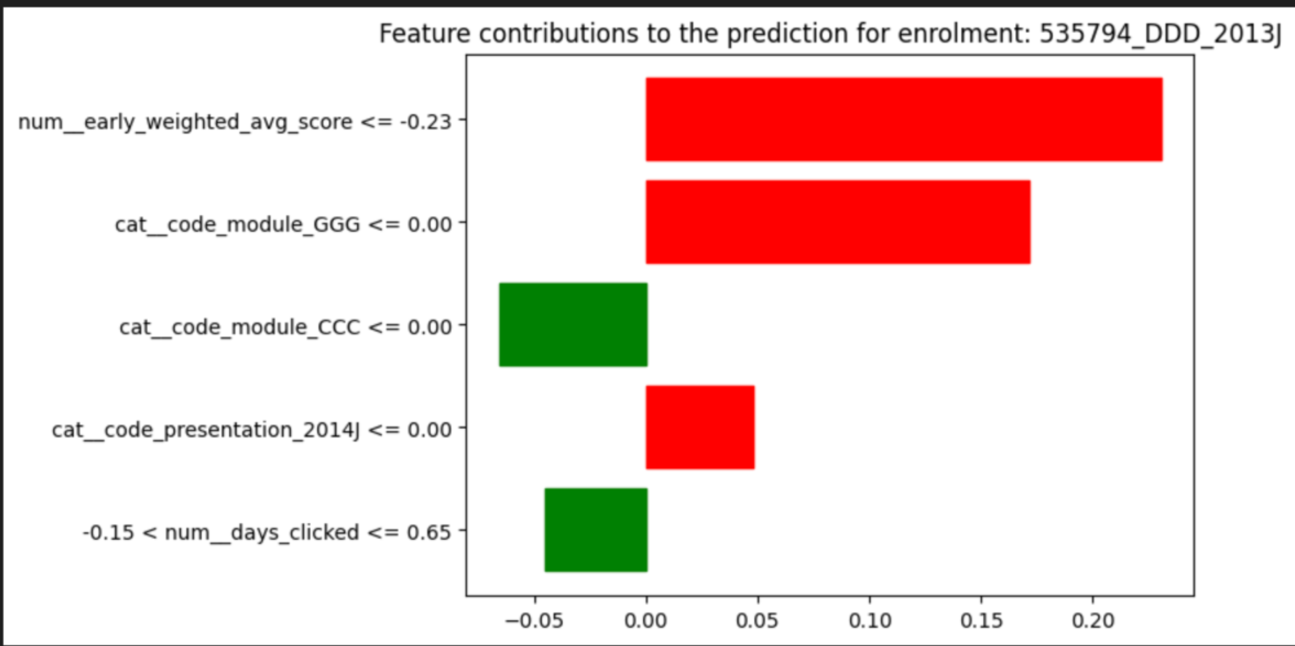
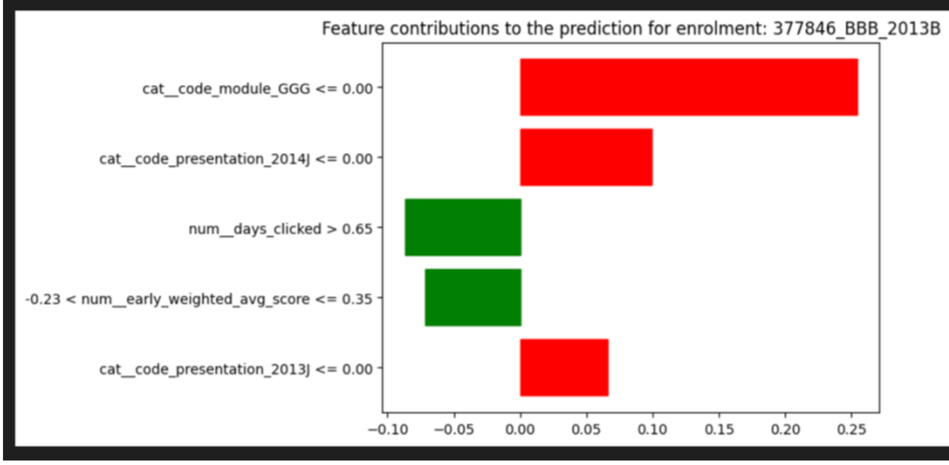


Figure 6: LIME application #2

```
Student Index: 575
Enrolment Instance ID: 377846_BBB_2013B
Predicted: Pass (42% probability of fail)
Actual:    Pass

Top Contributing Features:
- code_module_GGG: 0.00 (↑ Predictor of Fail)
- code_presentation_2014J: 0.00 (↑ Predictor of Fail)
- days_clicked: 2.33 (↓ Predictor of Pass)
- early_weighted_avg_score: -0.15 (↓ Predictor of Pass)
- code_presentation_2013J: 0.00 (↑ Predictor of Fail)
```



Part 7: References

References are progressive over Part A, B and C of this report.

Datasets

Open University Learning Analytics Dataset (OULAD)

Kuzilek, J, Hlosta, M & Zdrahal, Z 2017, 'Open University Learning Analytics dataset', *Scientific Data*, vol. 4, article 170171, doi:10.1038/sdata.2017.171, viewed 22 June 2025, https://analyse.kmi.open.ac.uk/open-dataset\

Universiti Malaya Student Engagement Dataset (SED)

Kassim, MSS, Azizul, ZH & Fuaad, AAH 2025, 'Student Engagement Dataset (SED): An online learning activity dataset', *IEEE Access*, early access, viewed 22 June 2025, https://ieeexplore.ieee.org/document/10844083\

University of California-Irvine Dataset (UCI)

Cortez, P & Silva, A 2008, 'Using data mining to predict secondary school student performance', *Proceedings of the European Simulation and Modelling Conference (EUROSIS)*, EUROSIS, Ghent, pp. 117–122, viewed 22 June 2025, https://www.researchgate.net/publication/228780408_Using_data_mining_to_predict_secondary_school_student_performance/

Other References

Huang, Q & Chen, J 2024, 'Enhancing academic performance prediction with temporal graph networks for massive open online courses', *Journal of Big Data*, vol. 11, no. 52, doi:10.1186/s40537-024-00918-5.

Alnasyan, B, Basheri, M, Alassafi, M *et al.* 2025, 'Kanformer: an attention-enhanced deep learning model for predicting student performance in virtual learning environments', *Social Network Analysis and Mining*, vol. 15, article 25, doi:10.1007/s13278-025-01446-7.

Khoudi, Z, Hafidi, N, Nachaoui, M *et al.* 2025, 'New approach to enhancing student performance prediction using machine learning techniques and clickstream data in virtual learning environments', *SN Computer Science*, vol. 6, article 139, doi:10.1007/s42979-024-03622-6.

Adnan, M, Ahmad, NH, Aziz, ME *et al.* 2021, 'Predicting at-risk students at different percentages of course length for early intervention using machine learning models', *IEEE Access*, vol. 9, pp. 7519–7539, doi:10.1109/ACCESS.2021.3050103.

Jihaoui, ME, El Kheir Abra, O & Mansouri, K 2025, 'A deep learning regression model for predicting students' performance with Shapley additive explanations-based interpretability', in *Proceedings of the 5th International Conference on Innovative Research in Applied Science, Engineering and Technology (IRASET)*, Fez, Morocco, 1–8, viewed 22 June 2025, https://ieeexplore.ieee.org/document/11008069/.

Mohammad, AS, Al-Kaltakchi, MTS, Alshehabi Al-Ani, J & Chambers, JA 2023, 'Comprehensive evaluations of student performance estimation via machine learning', *Mathematics*, vol. 11, no. 14, article 3153, doi:10.3390/math11143153.

Adefemi, KO & Mutanga, MB 2025, 'A robust hybrid CNN–LSTM model for predicting student academic performance', *Digital*, vol. 5, no. 2, article 16, doi:10.3390/digital5020016.

Australian Government Department of Education 2023, *Higher Education Provider Guidelines 2023*, Federal Register of Legislation, Canberra, viewed 22 June 2025, https://www.legislation.gov.au/F2023L00400. legislation.gov.au

University of Adelaide 2023, *Support for Students Policy*, University of Adelaide, Adelaide, viewed 22 June 2025, https://www.adelaide.edu.au/policies/5003/

Department of Education (Australian Government) 2024, Success rate – Glossary, Tertiary Collection of Student Information (TCSI), viewed 22 June 2025, https://www.tcsisupport.gov.au/glossary/glossaryterm/Success%20rate.

Department of Education (Australian Government) 2024, Retention rate – Glossary, Tertiary Collection of Student Information (TCSI), viewed 22 June 2025, https://www.tcsisupport.gov.au/glossary/glossaryterm/retention-rate.

Appendix 1: Research Questions

Primary

Within the first 25% of teaching time, how accurately and transparently can we predict which enrolments will result in a failed course outcome?

Sub-questions

- **Census-date early-warning accuracy:** What ROC-AUC, F1, precision and recall does the model achieve when trained and evaluated on records available up to the census cutoff (~25% of course progress) on the OULAD dataset?
- **Post-census performance gain:** How much does predictive accuracy improve when the training window is extended to 50% progress, and is the gain large enough to justify waiting beyond Census date?
- **Explainable-AI insights for intervention:** Using SHAP and Permutation Feature Importance, which behavioural, demographic and historic-grade features drive the risk scores prior to the census date and at 50%, and how can these insights guide targeted interventions?
- **Cross-dataset consistency:** When the same pipeline is applied separately to the SED and UCI student-performance datasets, how do accuracy metrics and key explanatory features compare with those from OULAD?

Appendix 2: Abbreviations

- **EDM** - Educational data mining
- **OULAD** - Open University Learning Analytics Dataset
- **UCI** - University of California-Irvine
- **SED** - Student Engagement Dataset (SED) - Universiti Malaya
- **VLE** - Virtual Learning Environment
- **LMS** - Learning Management System
- **CNN** - Convolutional Neural Network
- **LSTM** - Long Short-Term Memory
- **PFI** - Permutation Feature Importance
- **SHAP** - Shapley Additive Explanations
- **XAI** - Explainable Artificial Intelligence
- **APP-TGN** - Attention-based Personalised Propagation - Temporal Graph Network
- **GRU** - Gated Recurrent Unit

Llewy Smith
a1706836

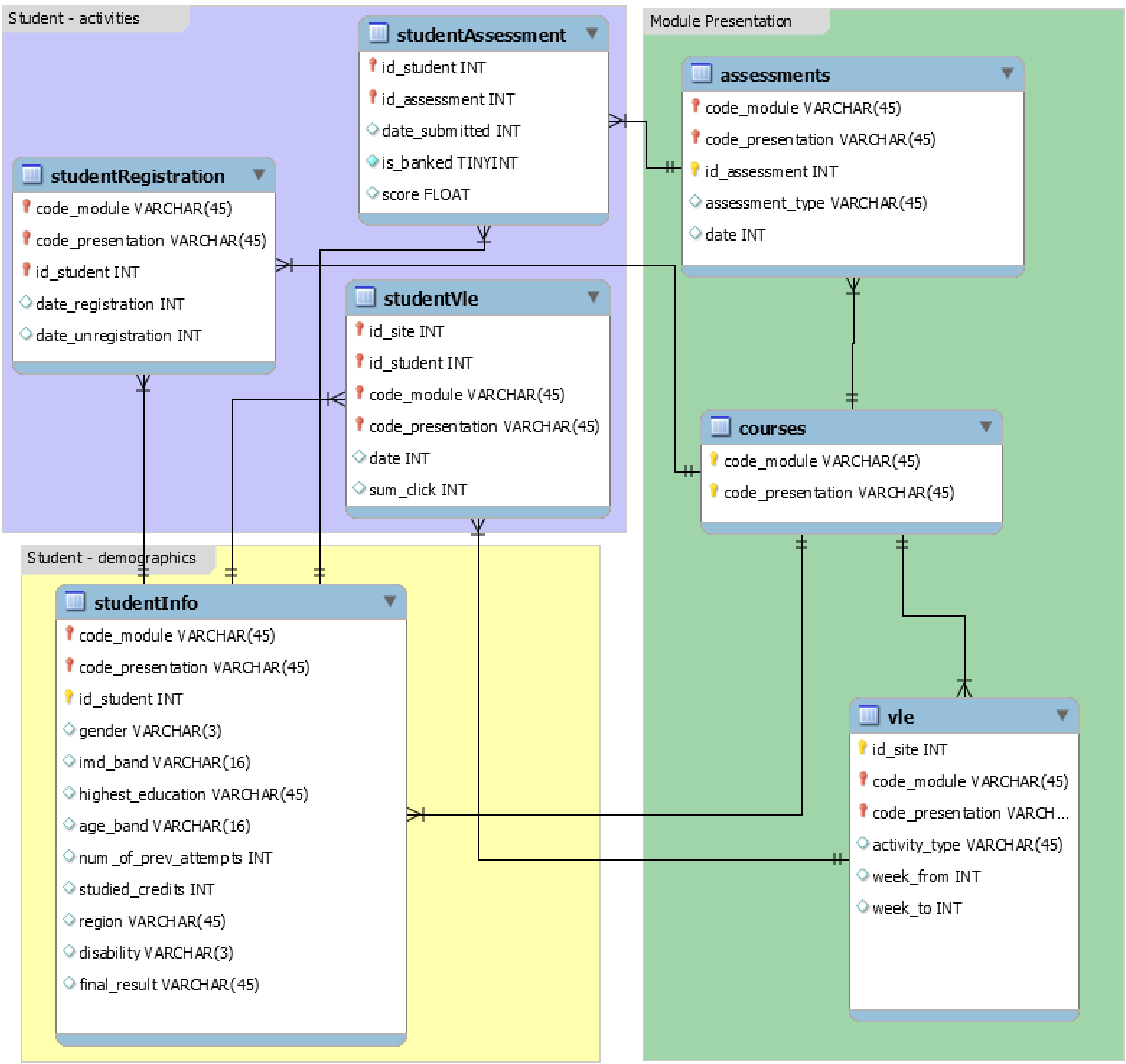
Appendix 3: Definitions

Success Rate – The Success rate for year(x) is the proportion of actual student load (EFTSL) for units of study that are passed divided by all units of study attempted (passed + failed + withdrawn) (Department of Education 2024).

Retention rate - Retention rate is a measure of the proportion of students who continue their studies after their first year. A more detailed definition is given here (Department of Education 2024).

Appendix 4: Dataset Schema

The full dataset schema is available publicly on the OULAD (2017) webpage, accessible here.



(OULAD, 2017)